

MODULE 16

Advanced Feature Engineering

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Advanced Feature Engineering

Why Feature Engineering?

Feature engineering is often more impactful than algorithm choice.

Good features make simple models work well.

Bad features make complex models fail.

Types:

Creation: derive new features from existing

Selection: pick most relevant features

Transformation: change scale or distribution

Extraction: learn features from raw data (DL)

Numerical Feature Engineering

```
import numpy as np, pandas as pd

# Log transform (fix right-skewed distributions)
df['log_price'] = np.log1p(df['price'])

# Power transform
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method='yeo-johnson')
df['price_pt'] = pt.fit_transform(df[['price']])

# Binning
df['age_bin'] = pd.cut(df['age'], bins=[0,18,35,60,100],
labels=['teen','young','middle','senior'])

# Polynomial features
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False, interaction_only=False)
X_poly = poly.fit_transform(X)
```

Categorical Feature Engineering

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OrdinalEncoder

# Target Encoding (mean of target per category)
df['city_target_enc'] = df.groupby('city')['price'].transform('mean')

# Frequency Encoding
df['city_freq'] = df['city'].map(df['city'].value_counts())

# Ordinal Encoding
ord_enc = OrdinalEncoder(categories=[['Low','Medium','High']])
df['size_ord'] = ord_enc.fit_transform(df[['size']])
```

```
# Binary Encoding
import category_encoders as ce
be = ce.BinaryEncoder(cols=['city'])
df_encoded = be.fit_transform(df)
```

Date and Time Features

```
import pandas as pd

df['date'] = pd.to_datetime(df['date'])

# Extract components
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['hour'] = df['date'].dt.hour
df['dayofweek'] = df['date'].dt.dayofweek
df['quarter'] = df['date'].dt.quarter
df['weekofyear'] = df['date'].dt.isocalendar().week

# Is weekend?
df['is_weekend'] = df['dayofweek'].isin([5, 6]).astype(int)

# Days since a reference date
df['days_since'] = (df['date'] - pd.Timestamp('2020-01-01')).dt.days
```

Automated Feature Engineering

```
import featuretools as ft

# Automated feature engineering
es = ft.EntitySet(id='customers')
es = es.add_dataframe(dataframe_name='customers', dataframe=customers_df,
index='customer_id')
es = es.add_dataframe(dataframe_name='orders', dataframe=orders_df, index='order_id',
time_index='order_date')
es = es.add_relationship('customers', 'customer_id', 'orders', 'customer_id')

feature_matrix, feature_names = ft.dfs(
entityset=es, target_dataframe_name='customers',
max_depth=2, agg_primitives=['mean', 'sum', 'count'],
trans_primitives=['month', 'year']
)
```